



Research Intake 2026

Day 4

Supervised Learning: Classification

A Beginner's Guide for Students

Understanding AI, Machine Learning, and Deep Learning

Gudsky Research Foundation

Table of Contents

1. Classification Algorithms
2. Regression Algorithms

CLASSIFICATION ALGORITHMS

1. Logistic Regression

Theory:

Logistic Regression is a supervised learning algorithm used for **binary classification** problems (Yes/No, 0/1, True/False). Despite its name, it's used for classification, not regression. It predicts the probability that an instance belongs to a particular class using the sigmoid function, which maps any real-valued number to a value between 0 and 1.

- Uses sigmoid/logistic function to squash output between 0 and 1
- Decision boundary: typically at 0.5 probability
- Can be extended to multi-class problems (one-vs-rest or softmax)

Real-World Example:

Email Spam Detection: You want to classify whether an email is spam (1) or not spam (0). Features might include:

- Number of exclamation marks
- Presence of words like "free", "win", "click here"
- Sender's email domain
- Email length

The algorithm learns weights for each feature and outputs a probability. If probability > 0.5 , classify as spam; otherwise, not spam.

Mathematical Equation:

Sigmoid Function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

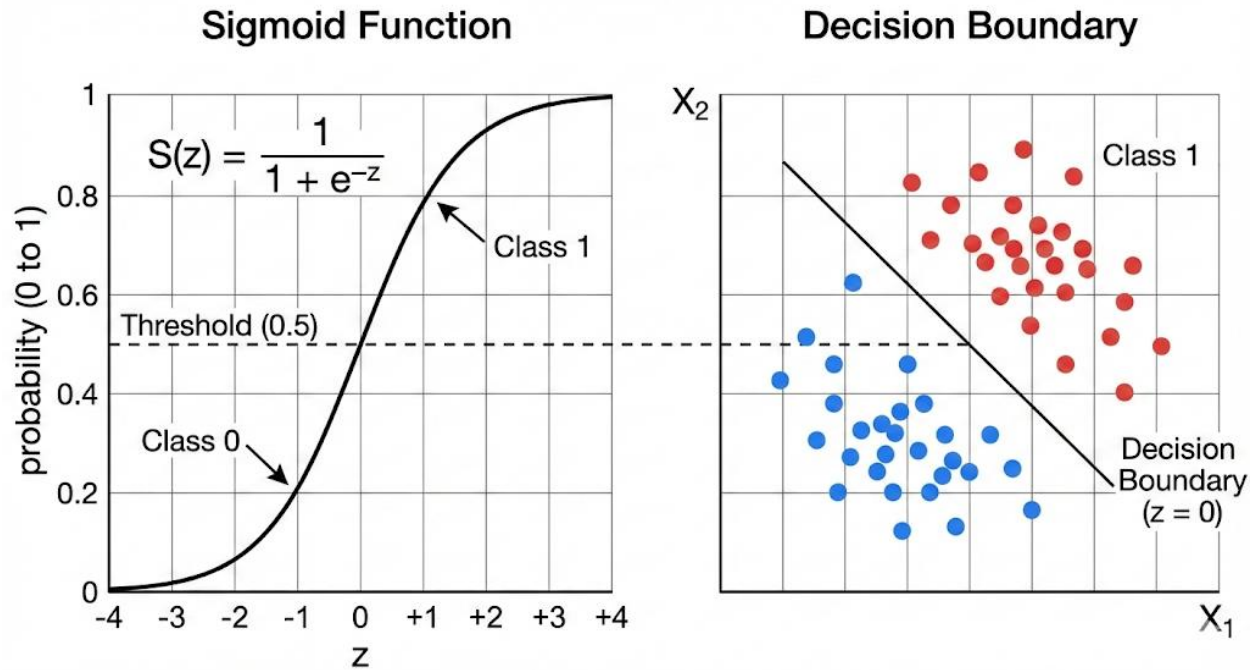
Prediction:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T\mathbf{x}$$

$$P(y=1|x) = \sigma(z) = 1 / (1 + e^{(-\mathbf{w}^T\mathbf{x})})$$

Cost Function (Log Loss):

$$J(\mathbf{w}) = -1/m \sum [y^{(i)} \log(h(\mathbf{x}^{(i)})) + (1-y^{(i)}) \log(1-h(\mathbf{x}^{(i)}))]$$



Where:

- w = weights
- x = features
- y = actual label (0 or 1)
- $h(x)$ = predicted probability
- m = number of training examples

2. Decision Trees

Theory:

A Decision Tree is a flowchart-like tree structure where:

- **Internal nodes** represent tests on features (e.g., "Age > 30?")
- **Branches** represent outcomes of tests
- **Leaf nodes** represent class labels

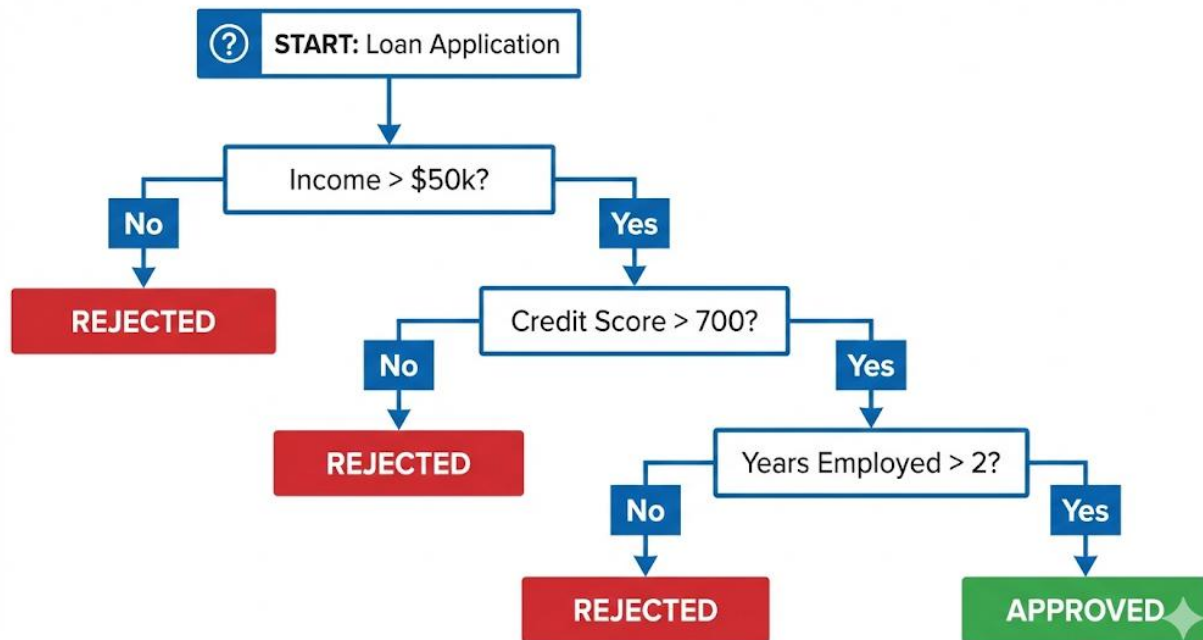
The tree is built using recursive partitioning, splitting data based on features that provide the most **information gain** (or least **Gini impurity**).

- Greedy algorithm (makes locally optimal choice at each split)

- Can handle both numerical and categorical data
- Non-parametric (doesn't assume data distribution)
- Prone to overfitting if not pruned

Real-World Example:

Loan Approval System: A bank decides whether to approve a loan based on:



Income > \$50,000?

- Yes → Check credit score
 - Credit score > 700? → **Approve**
 - Credit score ≤ 700? → Check employment years
- No → **Reject**

Each decision splits the data into smaller groups until we reach a final decision.

Mathematical Equation:

Gini Impurity (for classification):

$$\text{Gini} = 1 - \sum (p_i)^2$$

Information Gain (using Entropy):

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \sum (|S_i|/|S|) \times \text{Entropy}(S_i)$$

Best Split Selection:

$$\text{Best Feature} = \text{argmax}(\text{Information Gain})$$

Where:

- p_i = proportion of class i in the node
 - S = dataset at current node
 - S_i = subset after split
-

3. Random Forest

Random Forest is an **ensemble learning** method that creates multiple decision trees and combines their predictions. It's called "random" because:

1. Each tree is trained on a random subset of data (bootstrap sampling)
2. Each split considers only a random subset of features

Concepts:

- Bagging (Bootstrap Aggregating) technique
- Reduces overfitting compared to single decision trees
- Final prediction: majority vote (classification) or average (regression)
- Can estimate feature importance

Real-World Example:

Medical Diagnosis: Instead of relying on one doctor's opinion, imagine consulting 100 doctors (100 trees):

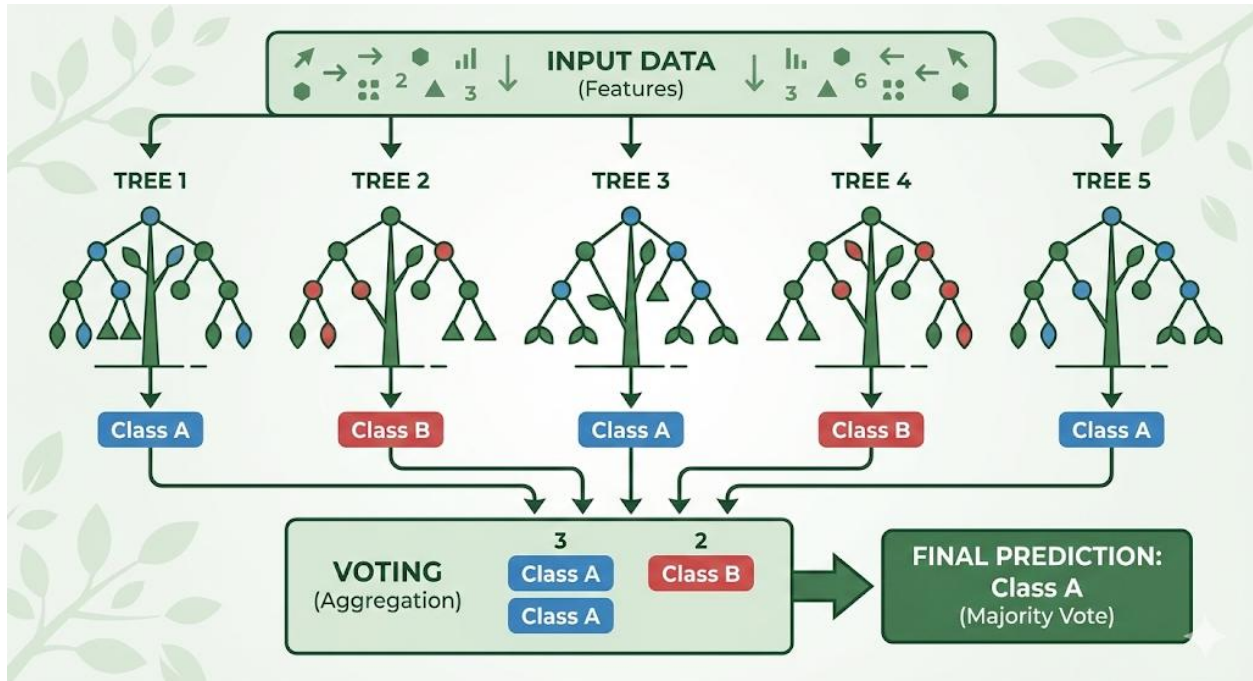
- Each doctor sees a slightly different patient history (bootstrap sample)
- Each doctor focuses on different symptoms (random features)
- Final diagnosis: what majority of doctors agree on

For diabetes prediction:

- Tree 1 focuses on: glucose level, BMI

- Tree 2 focuses on: age, blood pressure
- Tree 3 focuses on: family history, glucose
- ...and so on

If 70 out of 100 trees predict diabetes, the final prediction is "Diabetes".



Mathematical Equation:

Bootstrap Sampling:

For each tree $t = 1$ to T :

Create dataset D_t by sampling with replacement from D

Random Feature Selection:

At each node, randomly select m features from total M features

(typically $m = \sqrt{M}$ for classification)

Final Prediction (Classification):

$$\hat{y} = \text{mode} \{h_1(x), h_2(x), \dots, h_T(x)\}$$

Final Prediction (Regression):

$$\hat{y} = (1/T) \sum h_t(x)$$

Where:

- T = number of trees
 - $h_t(x)$ = prediction from tree t
 - m = features considered at each split
 - M = total features
-

4. Gradient Boosting (XGBoost, LightGBM, CatBoost)

Theory:

Gradient Boosting is an ensemble technique that builds trees **sequentially**. Each new tree tries to correct the errors made by the previous trees. Unlike Random Forest (parallel trees), Gradient Boosting builds trees one after another.

- Boosting: combining weak learners to create a strong learner
- Each tree learns from the mistakes of previous trees
- Uses gradient descent to minimize loss function
- Popular implementations: XGBoost, LightGBM, CatBoost

Real-World Example:

Student Exam Preparation: Think of preparing for an exam in iterations:

1. **First attempt (Tree 1):** You take a practice test and score 60/100
2. **Identify mistakes:** You got questions wrong in calculus (error = -40)
3. **Second attempt (Tree 2):** You focus on calculus errors, improving score to 75/100
4. **Identify remaining mistakes:** Still weak in probability (error = -25)
5. **Third attempt (Tree 3):** Focus on probability, score reaches 85/100
6. Continue until you reach target score

Each study session (tree) focuses on correcting previous mistakes (residuals).

Mathematical Equation:

Boosting Algorithm:

$F_0(x)$ = initial prediction (usually mean for regression)

For $m = 1$ to M :

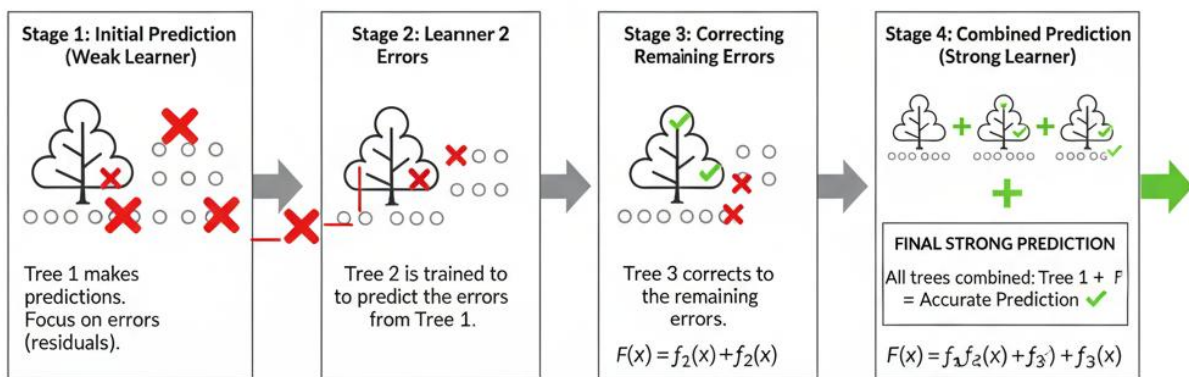
1. Compute residuals: $r_{im} = y_i - F_{m-1}(x_i)$
2. Fit tree $h_m(x)$ to residuals
3. Update model: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

Final Prediction:

$$\hat{y} = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \dots + \eta \cdot h_m(x)$$

Gradient Descent Form:

$$F_m(x) = F_{m-1}(x) - \eta \cdot \nabla L(y, F_{m-1}(x))$$



Where:

- η = learning rate (shrinkage parameter)
- M = number of trees
- r_{im} = residual for instance i at iteration m
- L = loss function

5. Support Vector Machines (SVM)

Theory:

SVM finds the optimal **hyperplane** that best separates different classes with the **maximum margin**. The margin is the distance between the hyperplane and the nearest data points from each class (called **support vectors**).

- Works well for high-dimensional data
- Kernel trick: can create non-linear decision boundaries
- Support vectors: data points closest to the decision boundary
- Common kernels: Linear, Polynomial, RBF (Radial Basis Function), Sigmoid

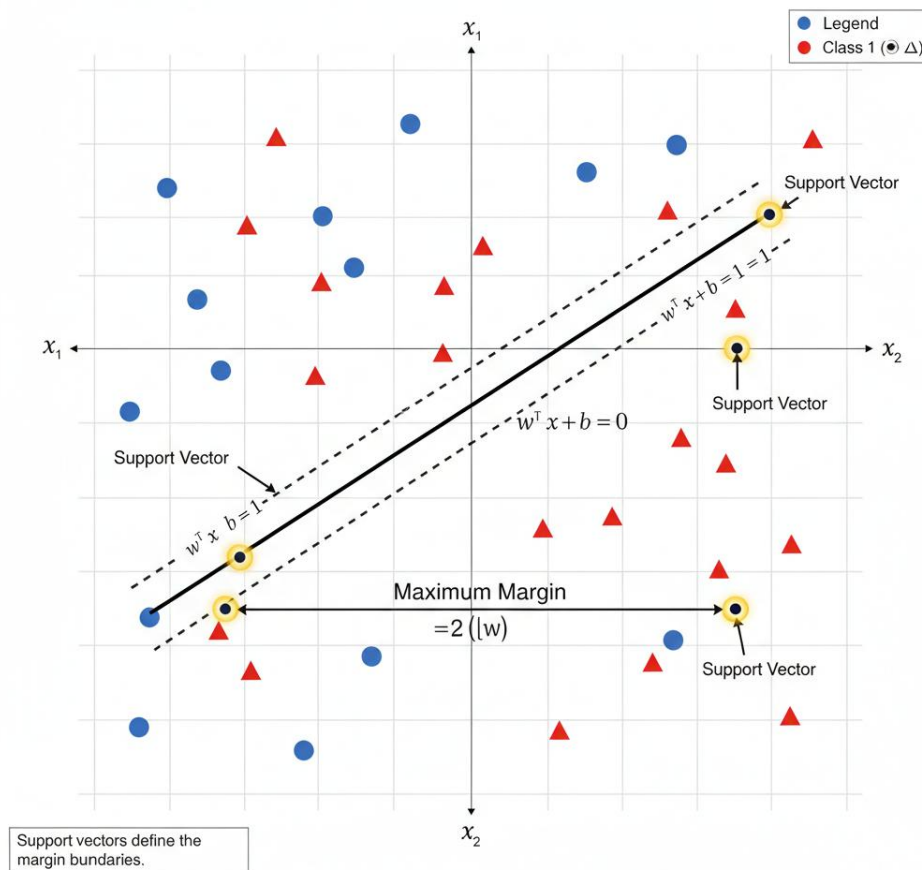
Real-World Example:

Customer Segmentation: Imagine separating customers who will buy a product (blue) from those who won't (red) based on age and income:

- SVM draws a line (in 2D) or plane (in 3D+) between the two groups
- It maximizes the "safety margin" - keeps the decision boundary as far as possible from both groups
- Support vectors are the customers closest to this boundary (the difficult cases)

Non-linear example: Separating fruits (apples vs oranges) where simple line won't work - SVM uses kernel trick to transform data into higher dimensions where they become separable.

Linear SVM Classification with Maximum Margin Separation



Mathematical Equation:

Linear SVM (Hard Margin):

$$\text{Minimize: } (1/2)\|w\|^2$$

$$\text{Subject to: } y_i(w^T x_i + b) \geq 1, \text{ for all } i$$

Soft Margin SVM (with slack variables):

$$\text{Minimize: } (1/2)\|w\|^2 + C \cdot \sum \xi_i$$

$$\text{Subject to: } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

Decision Function:

$$f(x) = \text{sign}(w^T x + b)$$

Kernel Function (RBF example):

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Where:

- w = weight vector (normal to hyperplane)
- b = bias
- C = regularization parameter
- ξ_i = slack variable (allows misclassification)
- γ = kernel parameter

6. K-Nearest Neighbors (KNN)

Theory:

KNN is a **lazy learning** algorithm that doesn't build a model during training. Instead, it stores all training data and makes predictions by finding the K closest training examples to the new data point. The class is determined by majority vote among these K neighbors.

Key Concepts:

- Non-parametric: no assumptions about data distribution
- Lazy learning: no training phase, all computation at prediction time
- Choice of K is crucial (small K = noisy, large K = smooth boundaries)

- Distance metric matters (Euclidean, Manhattan, etc.)

Real-World Example:

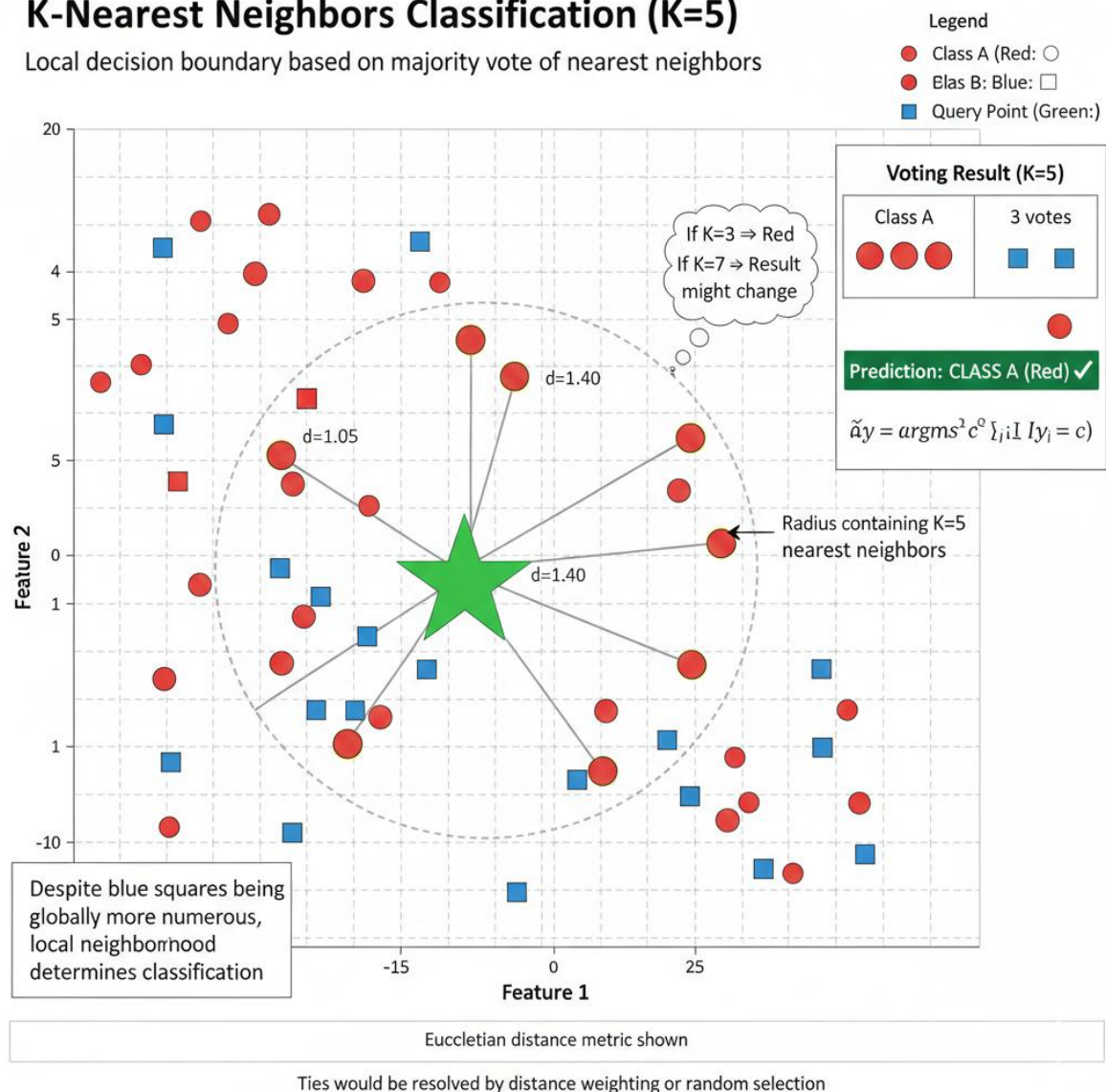
House Price Prediction (classification version: expensive/cheap): You want to classify if a house is expensive or cheap:

- Find K=5 most similar houses based on: size, bedrooms, location
- If 4 out of 5 similar houses are expensive → classify as expensive
- If 3 out of 5 similar houses are cheap → classify as cheap

Movie Recommendation: To predict if you'll like a movie, find 10 people most similar to you (based on past ratings), and check if they liked this movie.

K-Nearest Neighbors Classification (K=5)

Local decision boundary based on majority vote of nearest neighbors



Mathematical Equation:

Distance Calculation (Euclidean):

$$d(x, x_i) = \sqrt{(\sum (x_j - x_{ij})^2)}$$

Distance Calculation (Manhattan):

$$d(x, x_i) = \sum |x_j - x_{ij}|$$

Classification (Majority Vote):

$$\hat{y} = \text{mode} \{y_1, y_2, \dots, y_k\}$$

Weighted KNN:

$$\hat{y} = \text{argmax} \sum (w_i \cdot I(y_i = c))$$

$$\text{where } w_i = 1/d(x, x_i)^2$$

Where:

- K = number of neighbors
 - d = distance function
 - y_i = class of i th neighbor
 - w_i = weight (inverse of distance)
-

7. Naive Bayes

Theory:

Naive Bayes is a probabilistic classifier based on **Bayes' Theorem**. It's called "naive" because it assumes all features are **independent** of each other (which is rarely true in real life, but works surprisingly well).

Types:

- **Gaussian Naive Bayes:** Features follow normal distribution
- **Multinomial Naive Bayes:** For count data (text classification)
- **Bernoulli Naive Bayes:** For binary features

Real-World Example:

Email Spam Classification: Given an email with words: "free", "winner", "click"

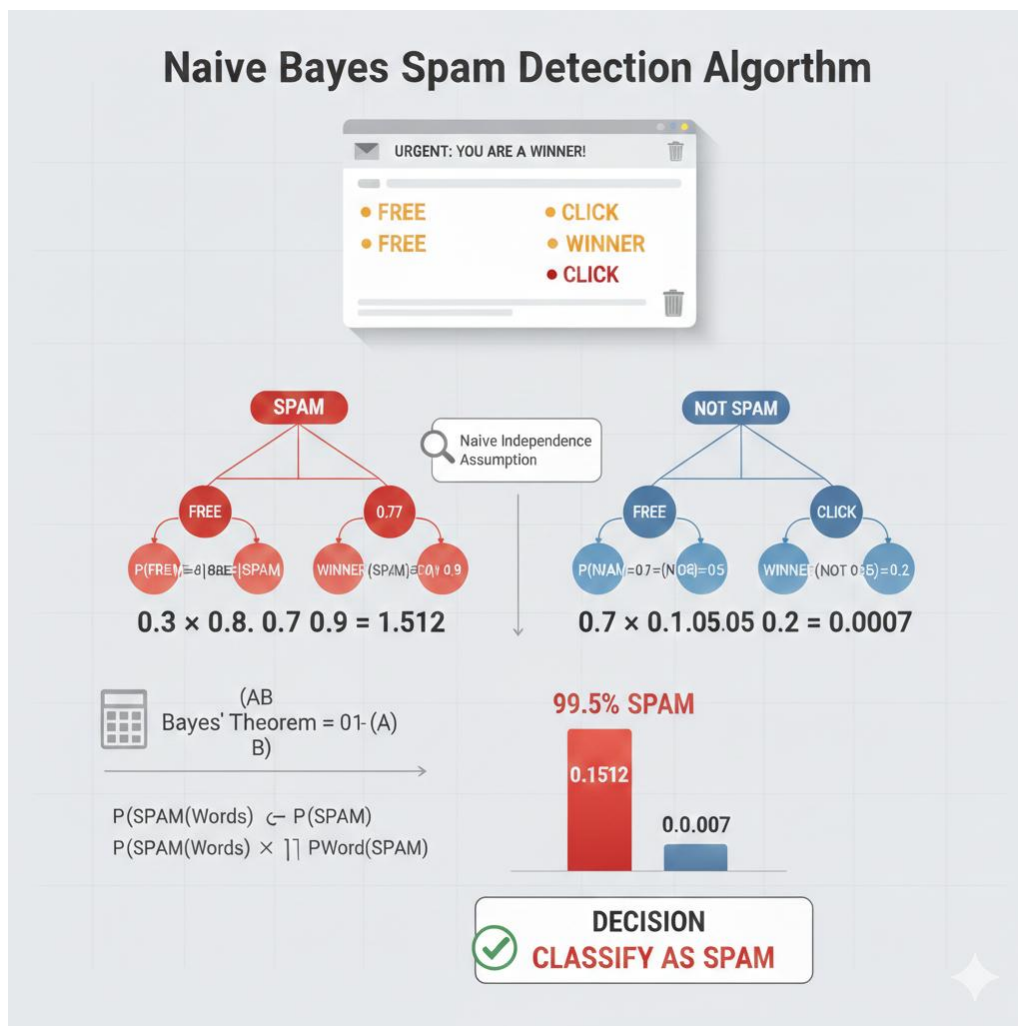
Calculate probability of spam:

- $P(\text{Spam}) = 0.3$ (30% of emails are spam)
- $P(\text{"free"}|\text{Spam}) = 0.8$
- $P(\text{"winner"}|\text{Spam}) = 0.7$
- $P(\text{"click"}|\text{Spam}) = 0.6$

Calculate probability of not spam:

- $P(\text{Not Spam}) = 0.7$
- $P(\text{"free"}|\text{Not Spam}) = 0.1$
- $P(\text{"winner"}|\text{Not Spam}) = 0.05$
- $P(\text{"click"}|\text{Not Spam}) = 0.2$

Using Bayes theorem, calculate which is more likely and classify accordingly.



Mathematical Equation:

Bayes' Theorem:

$$P(C|X) = P(X|C) \cdot P(C) / P(X)$$

Naive Bayes Assumption:

$$P(X|C) = P(x_1|C) \cdot P(x_2|C) \cdot \dots \cdot P(x_n|C)$$

Classification:

$$\hat{y} = \underset{C}{\operatorname{argmax}} P(C) \cdot \prod P(x_i|C)$$

Gaussian Naive Bayes (for continuous features):

$$P(x_i|C) = (1/\sqrt{2\pi\sigma_c^2}) \cdot \exp(-(x_i-\mu_c)^2/(2\sigma_c^2))$$

Where:

- C = class
 - X = feature vector
 - x_i = individual feature
 - μ_c = mean of feature for class C
 - σ_c = standard deviation for class C
-

8. AdaBoost (Adaptive Boosting)

Theory:

AdaBoost is an ensemble method that combines multiple **weak classifiers** to create a strong classifier. It works by:

1. Training a weak classifier on data
2. Increasing weights of misclassified samples
3. Training next classifier to focus on these hard examples
4. Repeating and combining all classifiers with weighted voting

Key Concepts:

- Adaptive: adjusts to errors of previous classifiers

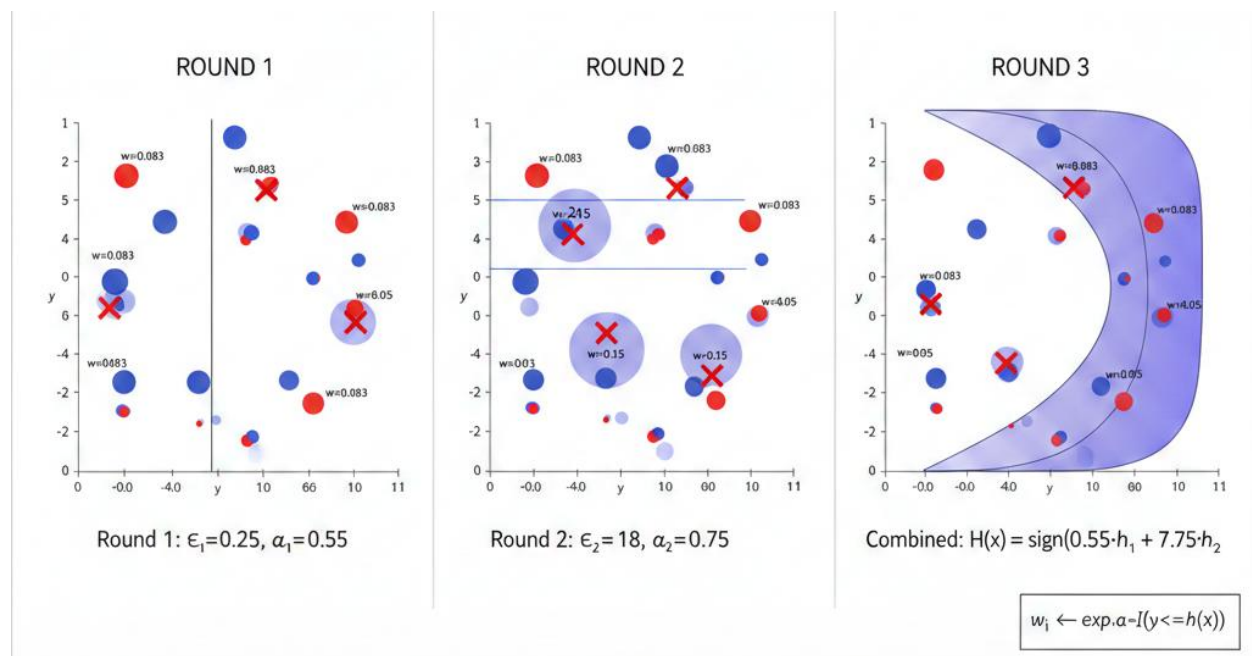
- Weak learners: typically decision stumps (1-level decision trees)
- Sample weighting: misclassified examples get higher importance
- Final model: weighted combination of all weak learners

Real-World Example:

Face Detection System: Detecting faces using simple features:

1. **Weak Classifier 1:** Checks if there's a dark horizontal region (eyes)
 - Gets 60% accuracy
 - Misclassified images get more attention
2. **Weak Classifier 2:** Checks for light vertical region (nose bridge)
 - Focuses on previously misclassified images
 - Gets 55% accuracy on hard cases
3. **Weak Classifier 3:** Checks for wider lower region (mouth)
 - Again focuses on mistakes
4. Combine all three: Now achieves 95% accuracy!

Each simple check (weak learner) alone is barely better than random guessing, but together they're powerful.



Mathematical Equation:

Algorithm:

1. Initialize weights: $w_i^{(1)} = 1/N$ for all samples

2. For $t = 1$ to T :

a) Train weak classifier h_t using weights $w_i^{(t)}$

b) Calculate error:

$$\varepsilon_t = \sum w_i^{(t)} \cdot I(y_i \neq h_t(x_i))$$

c) Calculate classifier weight:

$$\alpha_t = (1/2) \ln((1-\varepsilon_t)/\varepsilon_t)$$

d) Update sample weights:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$$

e) Normalize weights: $\sum w_i^{(t+1)} = 1$

Final Prediction:

$$H(x) = \text{sign}(\sum \alpha_t \cdot h_t(x))$$

Where:

- w_i = weight of sample i
- ε_t = weighted error of classifier t
- α_t = importance of classifier t
- $h_t(x)$ = prediction of weak classifier t

9. Linear Discriminant Analysis (LDA)

Theory:

LDA is a classification technique that finds a linear combination of features that best separates two or more classes. It assumes:

- Features follow a normal (Gaussian) distribution
- All classes have the same covariance matrix
- It maximizes the ratio of between-class variance to within-class variance

Key Concepts:

- Can be used for dimensionality reduction (like PCA)
- Works well when classes are well-separated
- Assumes Gaussian distribution
- Computes class means and shared covariance

Real-World Example:

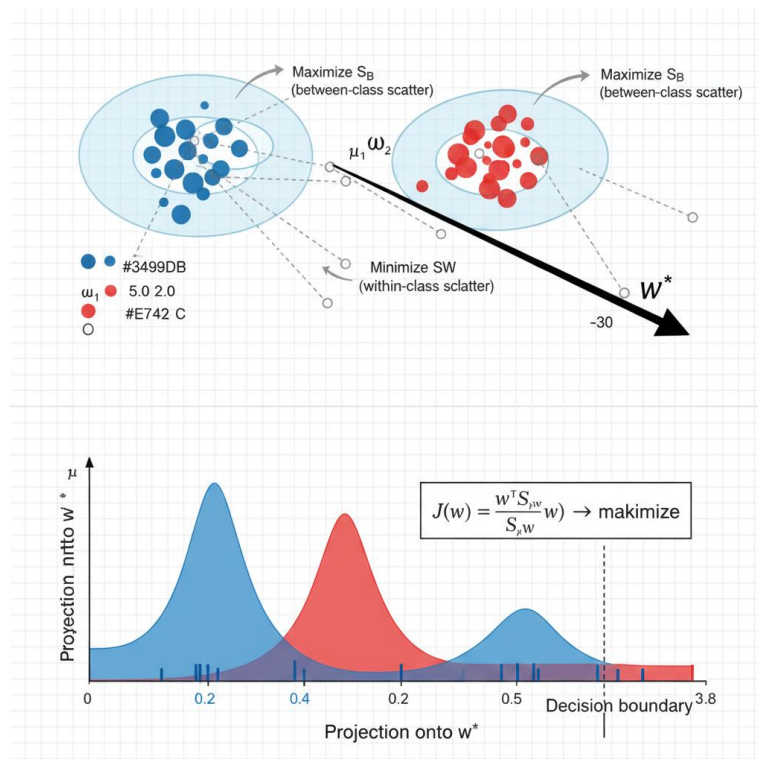
Wine Quality Classification: Classifying wines into categories (Low, Medium, High quality) based on:

- Alcohol content
- Acidity
- Sugar content
- pH level

LDA finds the best combination like:

$$\text{Quality Score} = 0.6 \times \text{Alcohol} + 0.3 \times \text{Acidity} - 0.2 \times \text{pH} + 0.4 \times \text{Sugar}$$

This score maximizes separation between quality classes. Wines with similar quality scores are grouped together.



Mathematical Equation:**Between-Class Scatter Matrix:**

$$S_\beta = \sum N_i(\mu_i - \mu)(\mu_i - \mu)^T$$

Within-Class Scatter Matrix:

$$S_v = \sum (x - \mu_i)(x - \mu_i)^T$$

LDA Objective:

$$\text{Maximize: } J(w) = (w^T S_\beta w) / (w^T S_v w)$$

Decision Boundary (2-class case):

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - (1/2) \mu_k^T \Sigma^{-1} \mu_k + \log(\pi_k)$$

Classify to class k if $\delta_k(x)$ is maximum

Where:

- μ_i = mean of class i
 - μ = overall mean
 - N_i = number of samples in class i
 - Σ = shared covariance matrix
 - π_k = prior probability of class k
-

REGRESSION ALGORITHMS

1. Linear Regression

Theory:

Linear Regression models the relationship between a dependent variable (target) and one or more independent variables (features) using a **linear equation**. It finds the best-fitting straight line (or hyperplane in multiple dimensions) through the data points by minimizing the sum of squared errors.

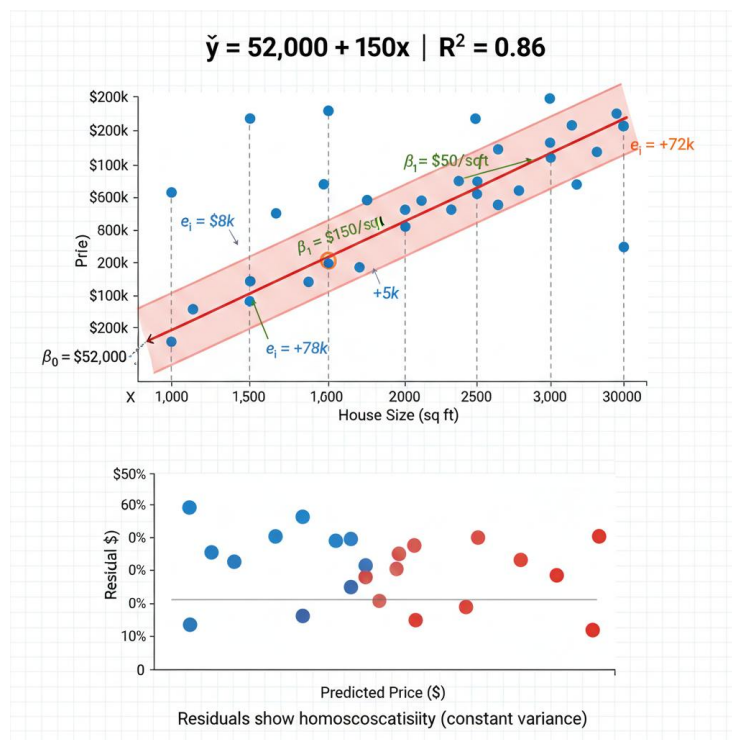
Types:

- **Simple Linear Regression:** One independent variable
- **Multiple Linear Regression:** Multiple independent variables

Real-World Example:

House Price Prediction: Predicting house price based on:

- Size in square feet
- Number of bedrooms
- Age of house
- Distance from city center



Model learns: $\text{Price} = 50,000 + 100 \times (\text{Size}) + 5,000 \times (\text{Bedrooms}) - 1,000 \times (\text{Age}) - 50 \times (\text{Distance})$

For a 2000 sq ft, 3-bedroom, 10-year-old house, 5 km from center:

$$\text{Price} = 50,000 + 100 \times 2000 + 5,000 \times 3 - 1,000 \times 10 - 50 \times 5$$

$$= 50,000 + 200,000 + 15,000 - 10,000 - 250$$

$$= \$254,750$$

Mathematical Equation:

Simple Linear Regression:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

Multiple Linear Regression:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Matrix Form:

$$y = X\beta + \varepsilon$$

Cost Function (Mean Squared Error):

$$J(\beta) = (1/2m) \sum (y_i - \hat{y}_i)^2 = (1/2m) \sum (y_i - (\beta_0 + \sum \beta_j x_{ij}))^2$$

Normal Equation (Closed Form Solution):

$$\beta = (X^T X)^{-1} X^T y$$

Gradient Descent Update:

$$\beta_j := \beta_j - \alpha \cdot (\partial J / \partial \beta_j)$$

Where:

- y = target variable
 - x = feature(s)
 - β = coefficients/weights
 - ε = error term
 - m = number of samples
 - α = learning rate
-

2. Polynomial Regression

Theory:

Polynomial Regression extends linear regression by modeling the relationship between variables as an **nth degree polynomial**. While still called "linear" regression (linear in parameters), it can capture non-linear relationships in data by creating polynomial features.

- Transforms features: $x \rightarrow x, x^2, x^3, \text{etc.}$
- Can fit curved relationships
- Higher degree = more flexible but risk of overfitting
- Still uses linear regression after feature transformation

Real-World Example:

Car Fuel Efficiency vs Speed: A car's fuel efficiency doesn't increase linearly with speed:

- At very low speeds (20 km/h): 15 km/L (inefficient)
- At moderate speeds (60 km/h): 25 km/L (most efficient)
- At high speeds (120 km/h): 18 km/L (wind resistance reduces efficiency)

This relationship is curved (parabolic), best captured by polynomial regression:

$$\text{Efficiency} = \beta_0 + \beta_1 \times (\text{Speed}) + \beta_2 \times (\text{Speed})^2$$

This creates an inverted U-shape curve matching real-world behavior.

Mathematical Equation:

Polynomial Regression (degree n):

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \beta_n X^n + \varepsilon$$

Multiple Variables with Polynomial Features:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \dots + \varepsilon$$

Feature Transformation:

$$\text{Original: } X = [x_1, x_2, \dots, x_n]$$

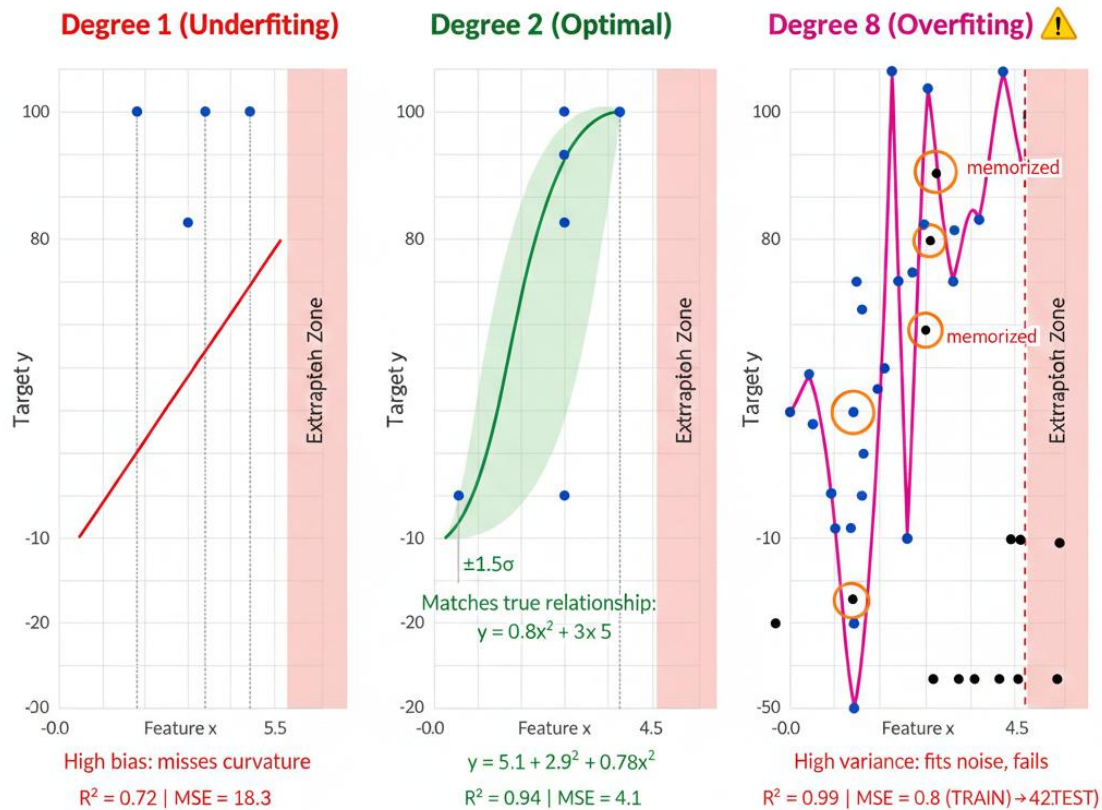
$$\text{Transformed: } X' = [1, x_1, x_2, x_1^2, x_1 x_2, x_2^2, x_1^3, \dots]$$

Same Cost Function as Linear Regression:

$$J(\beta) = (1/2m) \sum (y_i - \hat{y}_i)^2$$

Where:

- n = degree of polynomial
- β = coefficients for each polynomial term



Bias-Variance Tradeoff: Low complexity $\rightarrow$ high bias | High complexity $\rightarrow$ High variance

3. Ridge Regression (L2 Regularization)

Theory:

Ridge Regression is a **regularized** version of linear regression that adds a penalty term to prevent overfitting. It shrinks the coefficients by adding the **L2 norm** (sum of squared coefficients) to the cost function. This prevents any single feature from having too large an effect.

- Adds penalty proportional to square of coefficient magnitude
- Keeps all features but reduces their impact
- Good when you have many correlated features
- λ (lambda) controls regularization strength

Real-World Example:

Student Grade Prediction with Many Features: Predicting final grade using 50 features:

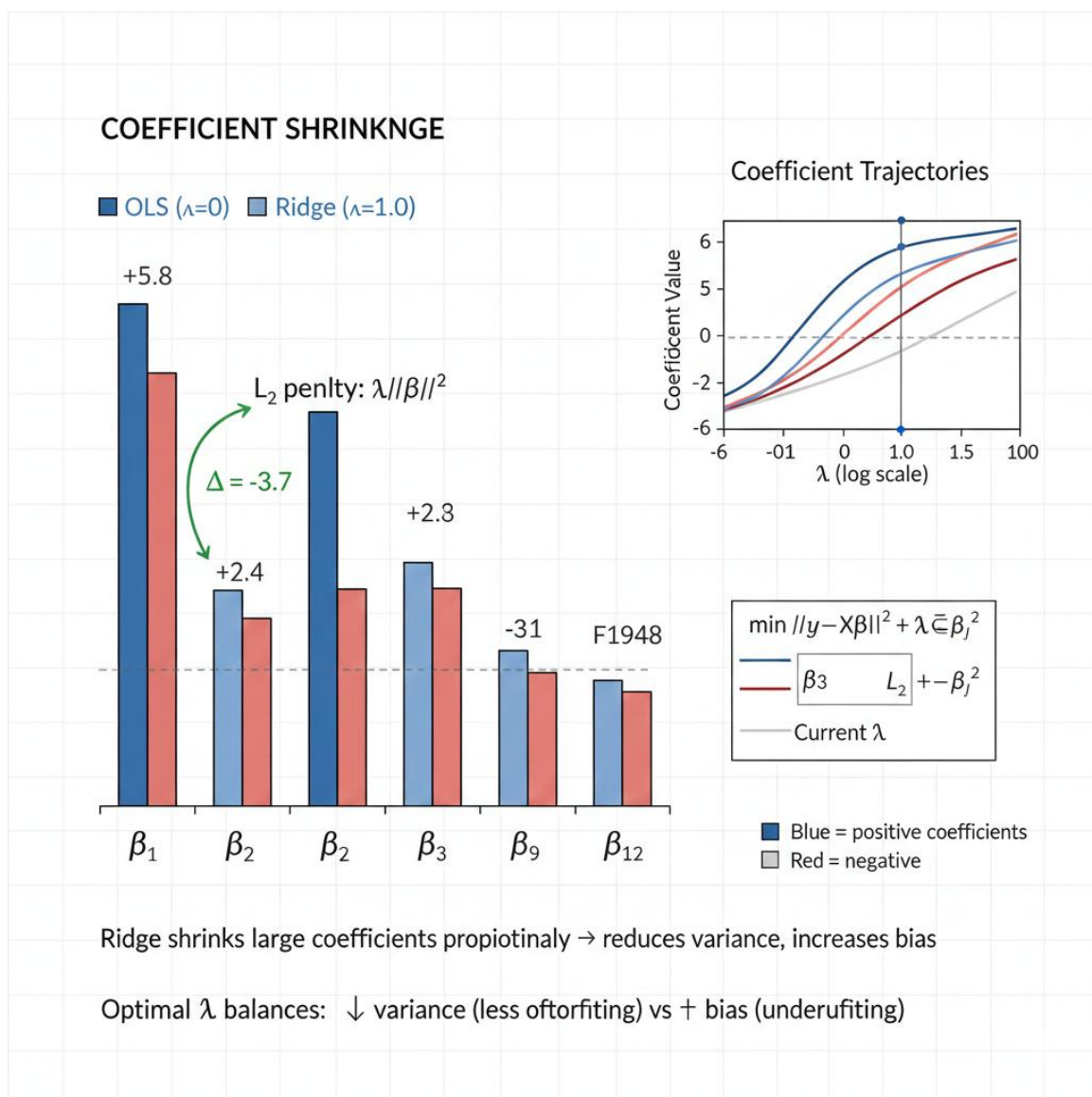
- Attendance, quiz scores, assignment scores, participation, study hours, sleep hours, etc.

Without regularization, the model might give huge weight to one feature (say, Quiz 1 score = +50 points) and negative huge weights to others (Assignment 3 = -45 points), leading to unstable predictions.

Ridge Regression prevents this by penalizing large coefficients, ensuring all features contribute reasonably. The model becomes:

$$\text{Grade} = 2 \times \text{Attendance} + 3 \times \text{AvgQuiz} + 1.5 \times \text{Study} - 0.5 \times \text{Absences} + \dots$$

(All coefficients stay moderate)



Mathematical Equation:

Ridge Regression Cost Function:

$$J(\beta) = (1/2m) \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

$$\frac{\sum (y_i - \hat{y}_i)^2}{2m} + \lambda \sum \beta_j^2$$

Original MSE L2 Penalty

Matrix Form:

$$J(\beta) = \|y - X\beta\|^2 + \lambda \|\beta\|^2$$

Closed Form Solution:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Gradient Descent Update:

$$\beta_j := \beta_j - \alpha \cdot [(1/m) \sum (\hat{y}_i - y_i) x_{ij} + (\lambda/m) \beta_j]$$

Where:

- λ = regularization parameter ($\lambda \geq 0$)
- Higher λ = more regularization (smaller coefficients)
- $\lambda = 0$ reduces to ordinary linear regression
- I = identity matrix

4. Lasso Regression (L1 Regularization)

Theory:

Lasso (Least Absolute Shrinkage and Selection Operator) Regression adds an **L1 penalty** (sum of absolute values of coefficients) to the cost function. Unlike Ridge, Lasso can **shrink some coefficients exactly to zero**, effectively performing **feature selection**.

- L1 penalty = sum of absolute coefficient values
- Can eliminate features (set coefficients to 0)
- Useful for feature selection
- Creates sparse models

Real-World Example:

Health Risk Prediction from Lifestyle Factors: Predicting heart disease risk from 30 lifestyle factors:

- Diet, exercise, smoking, alcohol, stress, sleep, genetics, etc.

Lasso might find that only 8 factors truly matter and set others to zero:

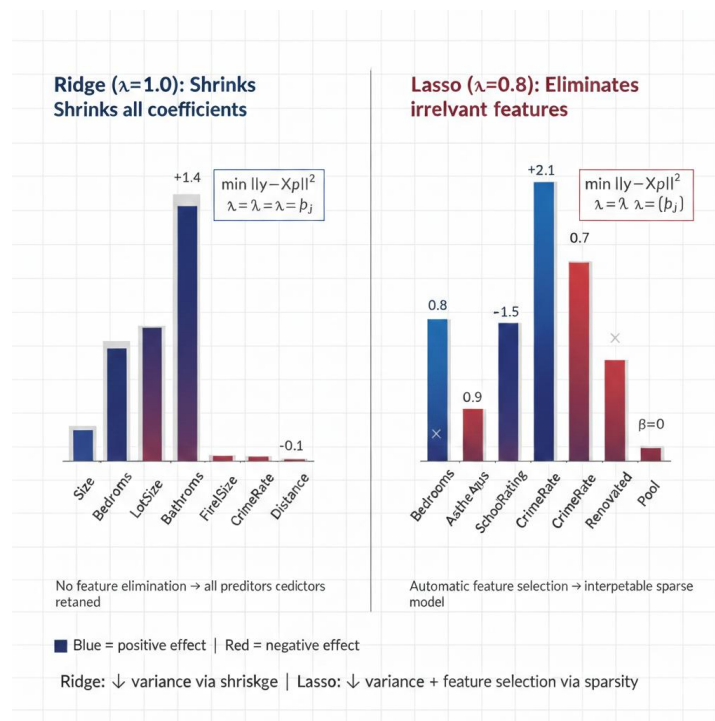
$$\text{Risk} = 0.5 \times \text{Smoking} + 0.3 \times \text{LDL_Cholesterol} + 0.2 \times \text{BloodPressure}$$

$$+ 0.15 \times \text{BMI} + 0.1 \times \text{Age} + 0.08 \times \text{FamilyHistory}$$

$$+ 0.05 \times \text{Exercise} + 0.03 \times \text{Stress}$$

$$+ 0 \times \text{Diet} + 0 \times \text{Sleep} + 0 \times \text{Alcohol} + \dots \text{ (22 features = 0)}$$

This tells doctors which factors to focus on!



Mathematical Equation:

Lasso Regression Cost Function:

$$J(\beta) = \underbrace{(1/2m) \sum (y_i - \hat{y}_i)^2}_{\text{Original MSE}} + \underbrace{\lambda \sum |\beta_j|}_{\text{L1 Penalty}}$$

Matrix Form:

$$J(\beta) = \|y - X\beta\|^2 + \lambda \|\beta\|_1$$

No Closed Form Solution (Use Optimization):

- Coordinate Descent
- Proximal Gradient Descent

Soft Thresholding Update:

$$\beta_j := \text{sign}(\beta_j^0) \cdot \max(|\beta_j^0| - \lambda/m, 0)$$

Where:

- λ = regularization parameter
 - Higher λ = more coefficients set to zero
 - $\|\beta\|_1$ = L1 norm (sum of absolute values)
 - β_j^0 = unregularized coefficient
-

5. Elastic Net

Theory:

Elastic Net combines **both L1 (Lasso) and L2 (Ridge)** penalties, getting benefits of both:

- Feature selection capability (from Lasso)
- Handling correlated features well (from Ridge)

It's useful when you have many features that are correlated with each other.

- Hybrid of Ridge and Lasso
- Two hyperparameters: λ (overall strength) and r (L1 ratio)
- $r = 0 \rightarrow$ Ridge, $r = 1 \rightarrow$ Lasso, $0 < r < 1 \rightarrow$ Elastic Net
- Best for high-dimensional data with correlations

Real-World Example:

Real Estate Price Prediction: Predicting house prices with highly correlated features:

- Square footage, number of rooms (correlated)
- Age of house, renovation year (correlated)
- Neighborhood income, school ratings (correlated)

Alternative Formulation:

$$J(\beta) = \|y - X\beta\|^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|^2$$

Penalty Term:

$$\text{Penalty} = \lambda[r \cdot \|\beta\|_1 + (1-r)/2 \cdot \|\beta\|^2]$$

Where:

- λ = overall regularization strength
 - r = L1 ratio ($0 \leq r \leq 1$)
 - $r = 0 \rightarrow$ Pure Ridge
 - $r = 1 \rightarrow$ Pure Lasso
 - $0 < r < 1 \rightarrow$ Elastic Net
 - $\lambda_1 = \lambda \cdot r$ (L1 parameter)
 - $\lambda_2 = \lambda \cdot (1-r)$ (L2 parameter)
-

6. Decision Tree Regressor

Theory:

Decision Tree Regressor works like classification trees but predicts continuous values instead of classes. At each leaf node, it predicts the **average** (or median) of training samples that reach that leaf.

Key Concepts:

- Splits based on minimizing variance or MSE
- Each leaf contains average of target values
- Non-linear, can capture complex patterns
- Prone to overfitting without pruning

Mathematical Equation:**Splitting Criterion (Minimize MSE):**

$$\text{MSE} = (1/n) \sum (y_i - \bar{y})^2$$

Information Gain for Regression:

$$\text{Gain} = \text{MSE}(\text{parent}) - \sum (n_i/n) \cdot \text{MSE}(\text{child}_i)$$

Best Split:

Best split = $\text{argmax}(\text{Gain})$

feature, threshold

Leaf Prediction:

$\hat{y} = (1/n) \sum y_i$ (average of samples in leaf)

Variance Reduction:

$VR = \text{Var}(S) - \sum(|S_i|/|S|) \cdot \text{Var}(S_i)$

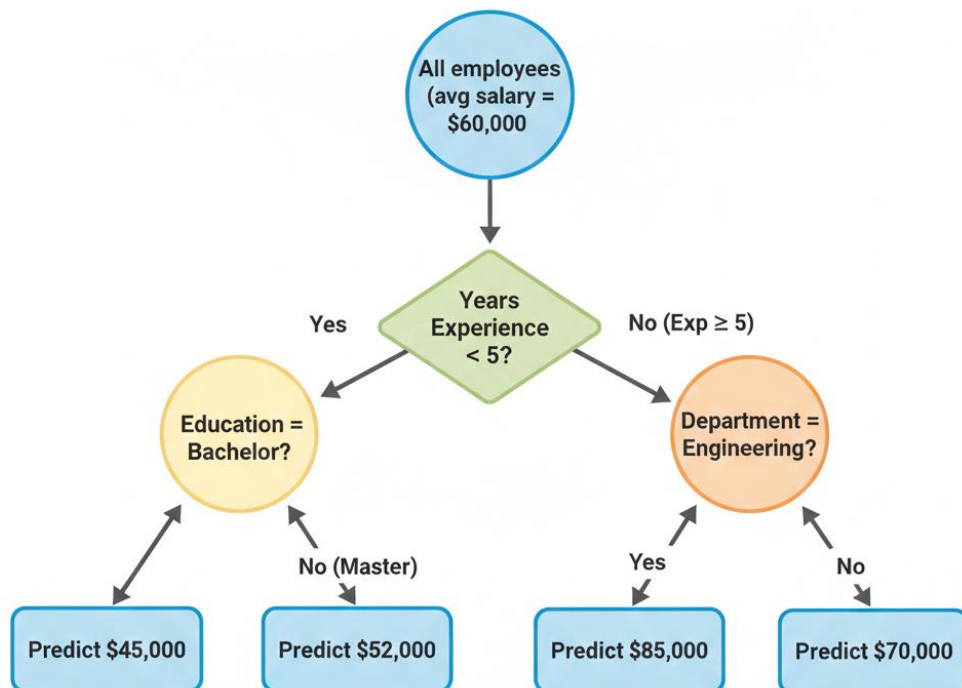
Where:

- $\bar{y}$ = mean of target values in node
- n = number of samples in node
- $\text{Var}(S)$ = variance of target in set S

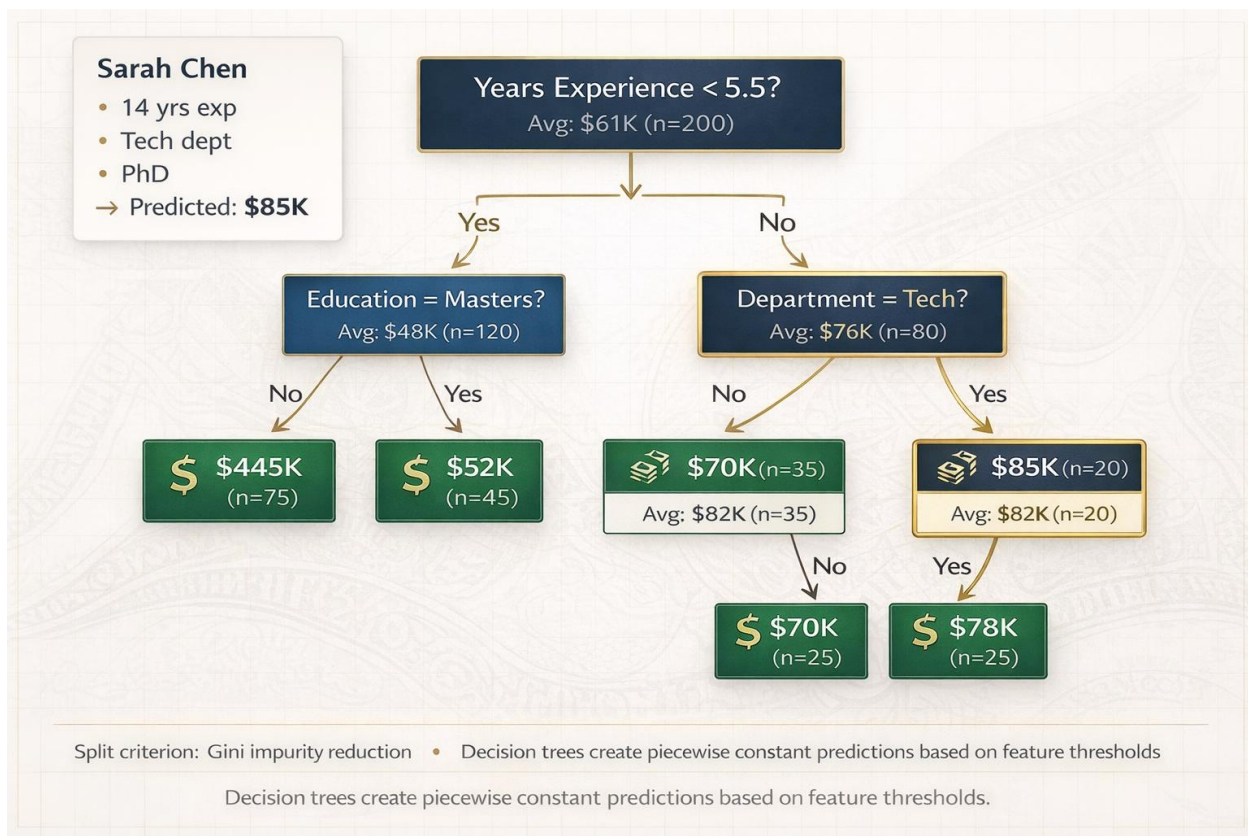
S_i = subset after split

Real-World Example:

Employee Salary Prediction



For new employee: 3 years exp, Bachelor → follows path → predicts \$45,000



7. Random Forest Regressor

Theory:

Random Forest Regressor extends Random Forest to regression tasks. It creates multiple decision trees and **averages their predictions** instead of voting.

- Ensemble of decision tree regressors
- Each tree trained on bootstrap sample
- Random feature selection at each split
- Final prediction = average of all trees
- Reduces overfitting compared to single tree

Real-World Example:

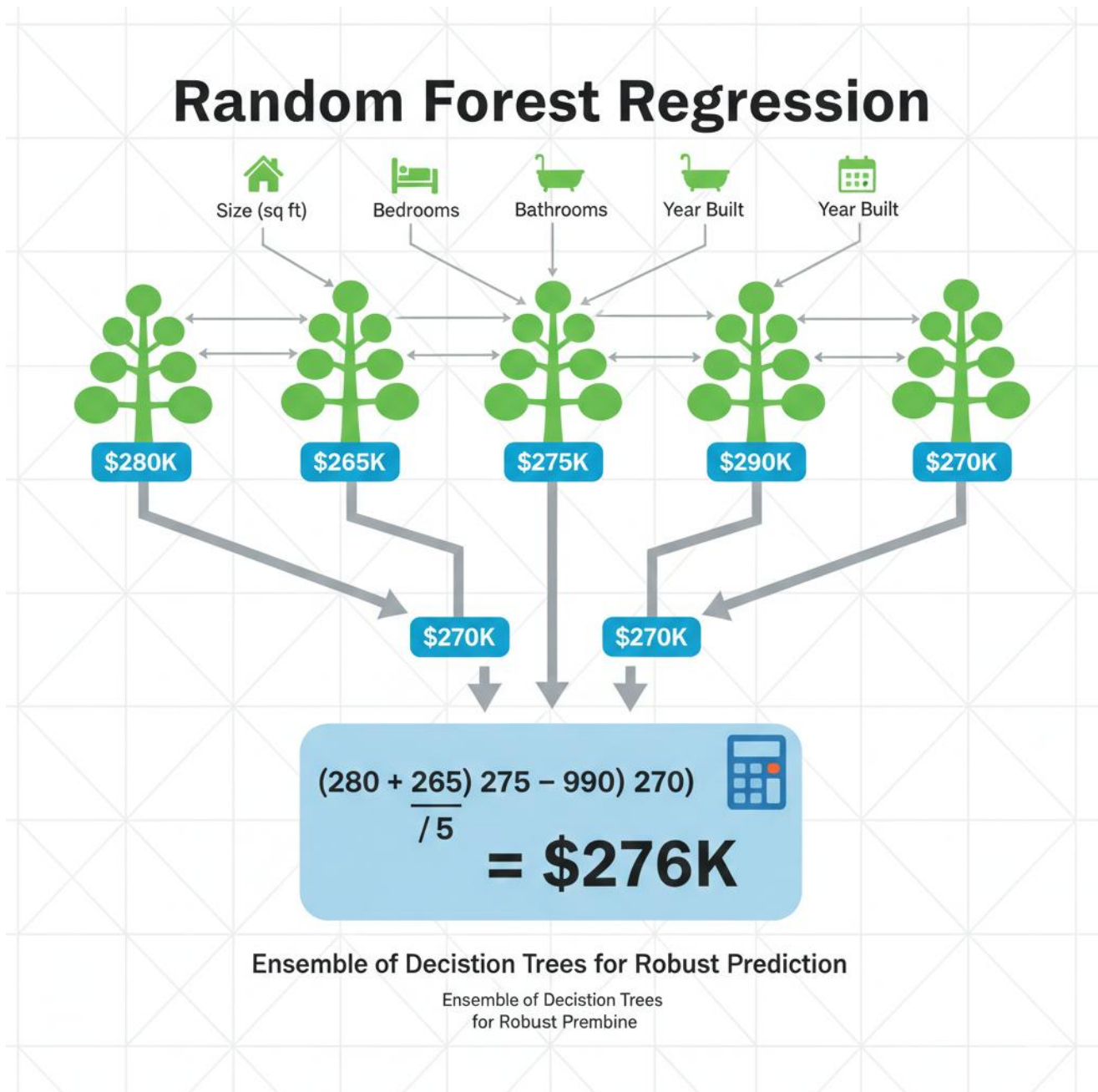
Real Estate Appraisal: Getting multiple appraisers (trees) to value a house:

- Appraiser 1 (focuses on location & size): \$280,000

- Appraiser 2 (focuses on age & condition): \$265,000
- Appraiser 3 (focuses on amenities & school): \$275,000
- ...
- Appraiser 100: \$270,000

Final price = Average of all 100 appraisers = \$272,000

Each appraiser sees slightly different aspects and data, but together they give a robust estimate.



Mathematical Equation:

Random Forest Regression:

For each tree $t = 1$ to T :

1. Create bootstrap sample D_t
2. Build tree $h_t(x)$ with random feature selection

Final Prediction (Average):

$$\hat{y} = (1/T) \sum h_t(x)$$

Out-of-Bag (OOB) Error:

$$\text{OOB_Error} = (1/n) \sum (y_i - \hat{y}_i^{\text{OOB}})^2$$

Feature Importance:

$$\text{Importance}(f_j) = (1/T) \sum \sum (\text{MSE_decrease at splits using } f_j)$$

Where:

- T = number of trees
- $h_t(x)$ = prediction from tree t
- D_t = bootstrap sample for tree t
- OOB = out-of-bag samples (not in bootstrap)

8. Gradient Boosting Regressor

Theory:

Gradient Boosting for regression builds trees sequentially, where each tree predicts the **residuals** (errors) of the previous trees. The final prediction is the sum of all tree predictions.

Key Concepts:

- Sequential tree building
- Each tree corrects previous trees' errors
- Uses gradient descent in function space
- Popular: XGBoost, LightGBM, CatBoost for regression

Real-World Example:

Stock Price Prediction:

1. **Initial guess (Tree 0):** Average stock price = \$100
2. **Actual prices:** [95, 105, 98, 110, 92]
3. **Residuals (errors):** [-5, +5, -2, +10, -8]
4. **Tree 1:** Learns to predict these residuals
 - Predictions: [-4, +4, -1, +9, -7]
5. **Updated prediction:** \$100 + Tree1 predictions
 - New predictions: [96, 104, 99, 109, 93]
6. **New residuals:** [-1, +1, -1, +1, -1]
7. **Tree 2:** Learns these smaller residuals
8. Continue until residuals become very small

Final prediction = Initial + 0.1×Tree1 + 0.1×Tree2 + 0.1×Tree3 + ...

Mathematical Equation:

Gradient Boosting Algorithm:

$F_0(x) = \bar{y}$ (initial prediction = mean)

For $m = 1$ to M :

1. Compute residuals:

$$r_{im} = y_i - F_{m-1}(x_i)$$

2. Fit regression tree $h_m(x)$ to residuals r_{im}

3. Update model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Final Prediction:

$$\hat{y} = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \dots + \eta \cdot h_m(x)$$

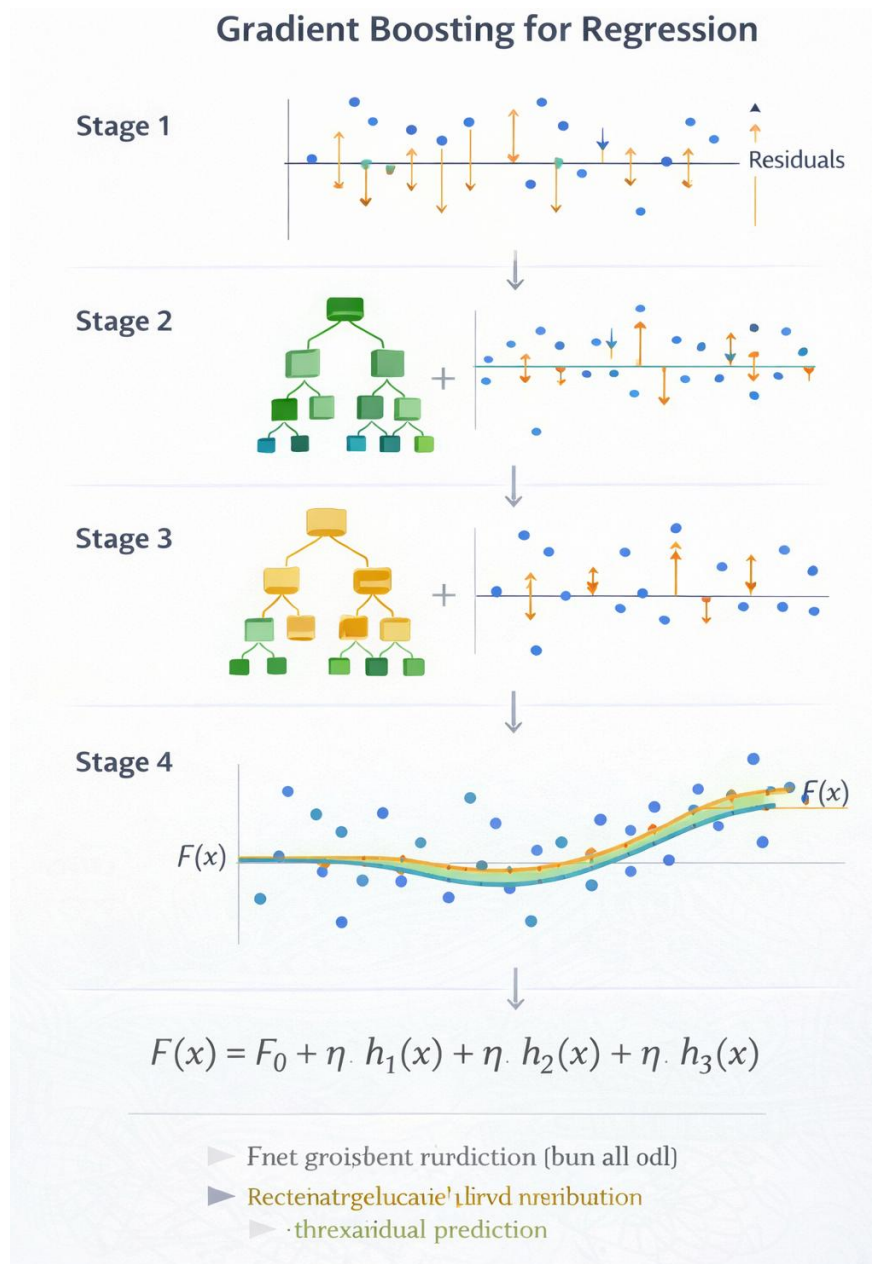
Loss Function (MSE):

$$L(y, F(x)) = (1/2)(y - F(x))^2$$

Gradient: $\partial L / \partial F = -(y - F(x)) = \text{-residual}$

Where:

- η = learning rate (shrinkage)
- M = number of trees
- $h_m(x)$ = tree at iteration m
- r_{im} = residual for sample i at iteration m



9. Support Vector Regression (SVR)

Theory:

SVR tries to fit the best line (or hyperplane) within a **margin of tolerance (ϵ)** around it. Unlike regular regression that minimizes error, SVR tries to keep errors within ϵ and only penalizes predictions outside this margin.

- ϵ -insensitive tube: errors $< \epsilon$ are ignored
- Support vectors: points outside the ϵ -tube
- Can use kernel trick for non-linear regression
- Robust to outliers (within margin)

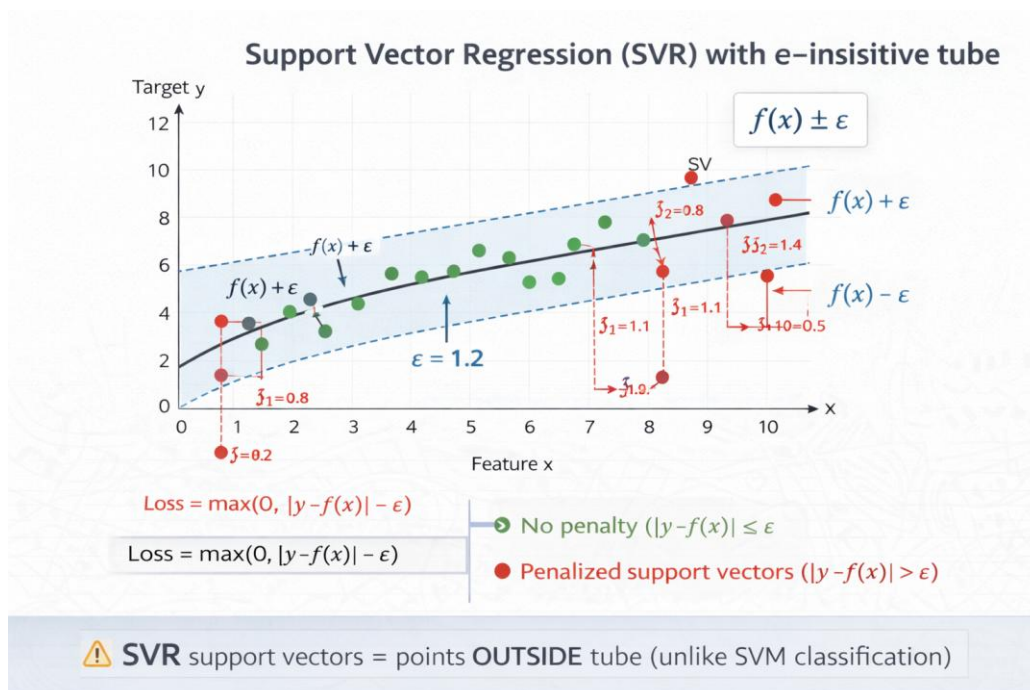
Real-World Example:

Temperature Prediction: You want to predict tomorrow's temperature. You're okay with errors up to $\pm 2^\circ\text{C}$ ($\epsilon = 2$).

Training data:

- Actual: 25°C , Predicted: 24°C $\rightarrow$ Error = 1°C $\rightarrow$ OK (within ϵ)
- Actual: 30°C , Predicted: 27°C $\rightarrow$ Error = 3°C $\rightarrow$ Penalize!
- Actual: 22°C , Predicted: 22.5°C $\rightarrow$ Error = 0.5°C $\rightarrow$ OK

SVR finds a prediction model where most errors are within $\pm 2^\circ\text{C}$, only caring about bigger errors.



Mathematical Equation:

SVR Objective (ϵ -insensitive loss):

$$\text{Minimize: } (1/2)\|w\|^2 + C \cdot \sum(\xi_i + \xi_i^*)$$

Subject to:

$$y_i - (w^T x_i + b) \leq \epsilon + \xi_i$$

$$(w^T x_i + b) - y_i \leq \epsilon + \xi_i^*$$

$$\xi_i, \xi_i^* \geq 0$$

Prediction Function:

$$f(x) = w^T x + b = \sum(\alpha_i - \alpha_i^*)K(x_i, x) + b$$

ϵ -insensitive Loss:

$$L_{\epsilon}(y) = \begin{cases} 0, & \text{if } |y - f(x)| \leq \epsilon \\ |y - f(x)| - \epsilon, & \text{otherwise} \end{cases}$$

RBF Kernel:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Where:

- ϵ = tolerance margin
- C = regularization parameter
- ξ_i, ξ_i^* = slack variables
- w = weight vector
- b = bias
- K = kernel function

10. K-Nearest Neighbors Regressor

Theory:

KNN Regressor predicts the target value as the **average** (or weighted average) of the K nearest neighbors' target values. It's the regression version of KNN classifier.

- Lazy learning (no training phase)
- Prediction = average of K nearest neighbors
- Distance-based (Euclidean, Manhattan, etc.)
- Can use distance-weighted averaging
- Sensitive to feature scaling

Real-World Example:

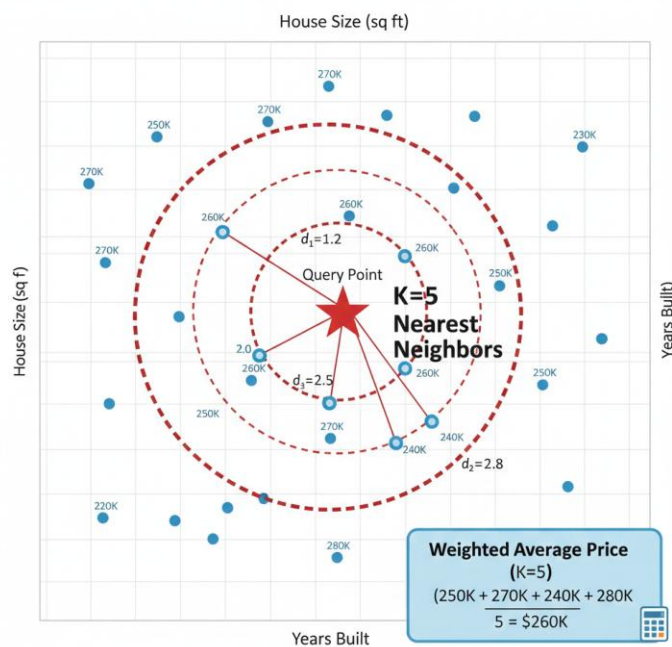
House Price Prediction: You want to predict the price of a 2000 sq ft house with 3 bedrooms.

Find K=5 most similar houses:

1. 1950 sq ft, 3 bed → \$250,000
2. 2100 sq ft, 3 bed → \$270,000
3. 2000 sq ft, 3 bed → \$260,000
4. 1980 sq ft, 2 bed → \$240,000
5. 2050 sq ft, 4 bed → \$280,000

Simple average: Price = $(250k + 270k + 260k + 240k + 280k) / 5 = \$260,000$

Distance-weighted: Closer houses get more weight in the average.



KNN Regression averages the target values of the K nearest data points.

Mathematical Equation:

Distance Calculation (Euclidean):

$$d(x, x_i) = \sqrt{(\sum (x_j - x_{ij})^2)}$$

Simple KNN Regression (Uniform Weights):

$$\hat{y} = (1/K) \sum y_i$$

$$i \in NK(x)$$

Distance-Weighted KNN:

$$\hat{y} = \sum w_i \cdot y_i / \sum w_i$$

$$\text{where } w_i = 1/d(x, x_i)^2$$

Alternative Weighting:

$$w_i = \exp(-d(x, x_i)^2/\sigma^2)$$

Where:

- K = number of neighbors
- $NK(x)$ = set of K nearest neighbors
- y_i = target value of neighbor i
- d = distance function
- w_i = weight of neighbor i
- σ = bandwidth parameter

11. Bayesian Ridge Regression

Theory:

Bayesian Ridge Regression is a probabilistic approach to Ridge Regression. Instead of finding a single best set of coefficients, it finds a **probability distribution** over coefficients. It automatically determines the regularization parameter λ from data.

- Treats coefficients as random variables
- Provides uncertainty estimates (confidence intervals)
- Automatically tunes regularization strength

- Based on Bayesian inference
- Prior: Gaussian distribution on coefficients

Real-World Example:

Medical Dosage Prediction: Predicting optimal drug dosage based on patient weight, age, kidney function:

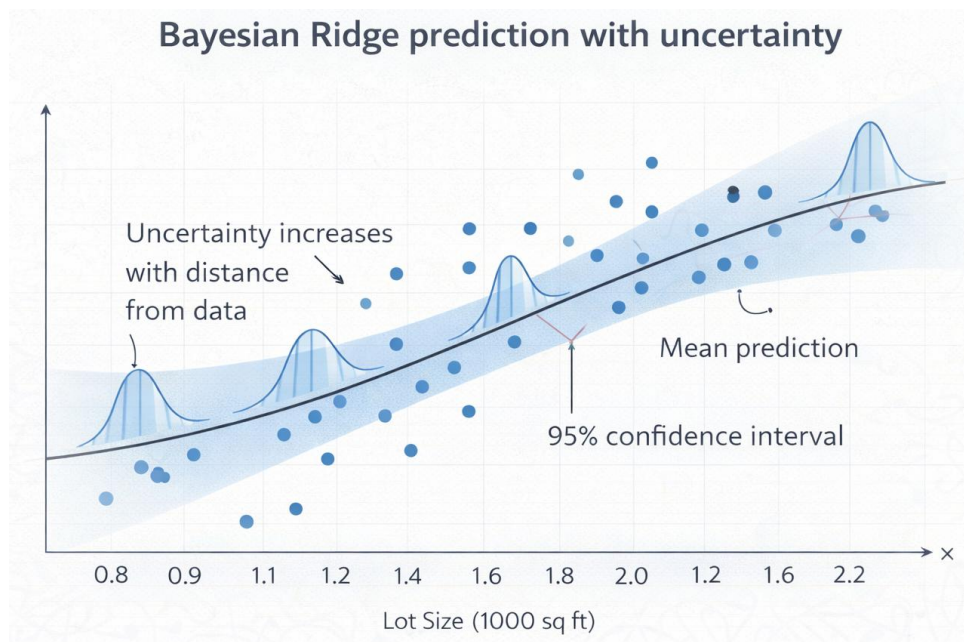
Instead of saying "dosage = 5.2 mg," Bayesian Ridge says:

- "dosage = 5.2 mg with 95% confidence interval [4.8, 5.6] mg"

This is crucial in medicine where:

- You need uncertainty estimates
- Different patients might need different doses
- Safety margins are important

The model learns both the prediction AND how confident it is about that prediction.



Mathematical Equation:

Bayesian Framework:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

Prior on Weights:

$$p(w|\alpha) = N(w|0, \alpha^{-1}I)$$

Likelihood:

$$p(y|X, w, \beta) = N(y|Xw, \beta^{-1}I)$$

Posterior Distribution:

$$p(w|y, X, \alpha, \beta) = N(w|\mu, \Sigma)$$

where:

$$\mu = \beta \Sigma X^T y$$

$$\Sigma = (\alpha I + \beta X^T X)^{-1}$$

Prediction Distribution:

$$p(y^*|x^*, X, y) = N(y^*|\mu^T x^*, \sigma^2(x^*))$$

$$\sigma^2(x^*) = 1/\beta + x^{*T} \Sigma x^*$$

Where:

- α = precision of prior ($1/\sigma_{\text{prior}}^2$)
 - β = precision of noise ($1/\sigma_{\text{noise}}^2$)
 - μ = posterior mean of weights
 - Σ = posterior covariance
 - N = normal distribution
-

12. Huber Regression

Theory:

Huber Regression is **robust to outliers**. It combines the best of two worlds:

- Behaves like **L2 (squared) loss** for small errors (smooth, efficient)
- Behaves like **L1 (absolute) loss** for large errors (robust to outliers)

It uses a threshold δ to decide when to switch between the two.

Key Concepts:

- Robust to outliers in target variable

- Less sensitive to extreme values than OLS
- Quadratic for small errors, linear for large errors
- Parameter δ controls transition point

Real-World Example:

Salary Prediction with Outliers: Predicting salaries based on experience:

- Most employees: \$40K - \$90K (normal range)
- Few CEOs: \$500K+ (outliers)

Linear Regression: Tries hard to fit outliers, messes up predictions for normal employees.

Huber Regression: Says "CEO salaries are outliers, don't let them dominate the model." It fits the normal range well while being less affected by extreme values.

For regular employees (error $< \delta$): Uses squared loss (smooth fitting) For CEO data (error $> \delta$): Uses absolute loss (less influence)

Mathematical Equation:

Huber Loss Function:

$$L_{\delta}(r) = \begin{cases} (1/2)(y - f(x))^2, & \text{if } |y - f(x)| \leq \delta \\ \delta|y - f(x)| - (1/2)\delta^2, & \text{otherwise} \end{cases}$$

where $r = y - f(x)$ (residual)

Cost Function:

$$J(w) = \sum L_{\delta}(y_i - w^T x_i)$$

Gradient:

$$\partial L / \partial r = \begin{cases} r, & \text{if } |r| \leq \delta \\ \delta \cdot \text{sign}(r), & \text{otherwise} \end{cases}$$

Comparison:

- $\delta \rightarrow \infty$: Behaves like L2 (MSE)
- $\delta \rightarrow 0$: Behaves like L1 (MAE)

- Typical: $\delta = 1.35\sigma$ (where σ is std dev of residuals)

Where:

- δ = threshold parameter
 - r = residual (error)
 - $\text{sign}(r) = +1$ if $r > 0$, -1 if $r < 0$
-

13. RANSAC (RANDOM Sample Consensus)

Theory:

RANSAC is a robust regression method that works when data contains many **outliers**. Instead of trying to fit all data, it:

1. Randomly samples small subsets
2. Fits model to each subset
3. Tests which model has most inliers (points fitting well)
4. Returns best model based on inliers only

Key Concepts:

- Iterative algorithm
- Robust to large proportion of outliers
- Non-deterministic (random sampling)
- Good for data with >50% outliers

Real-World Example:

Line Detection in Noisy Images: Finding a lane line on a road with many distracting features (other cars, road markings, shadows):

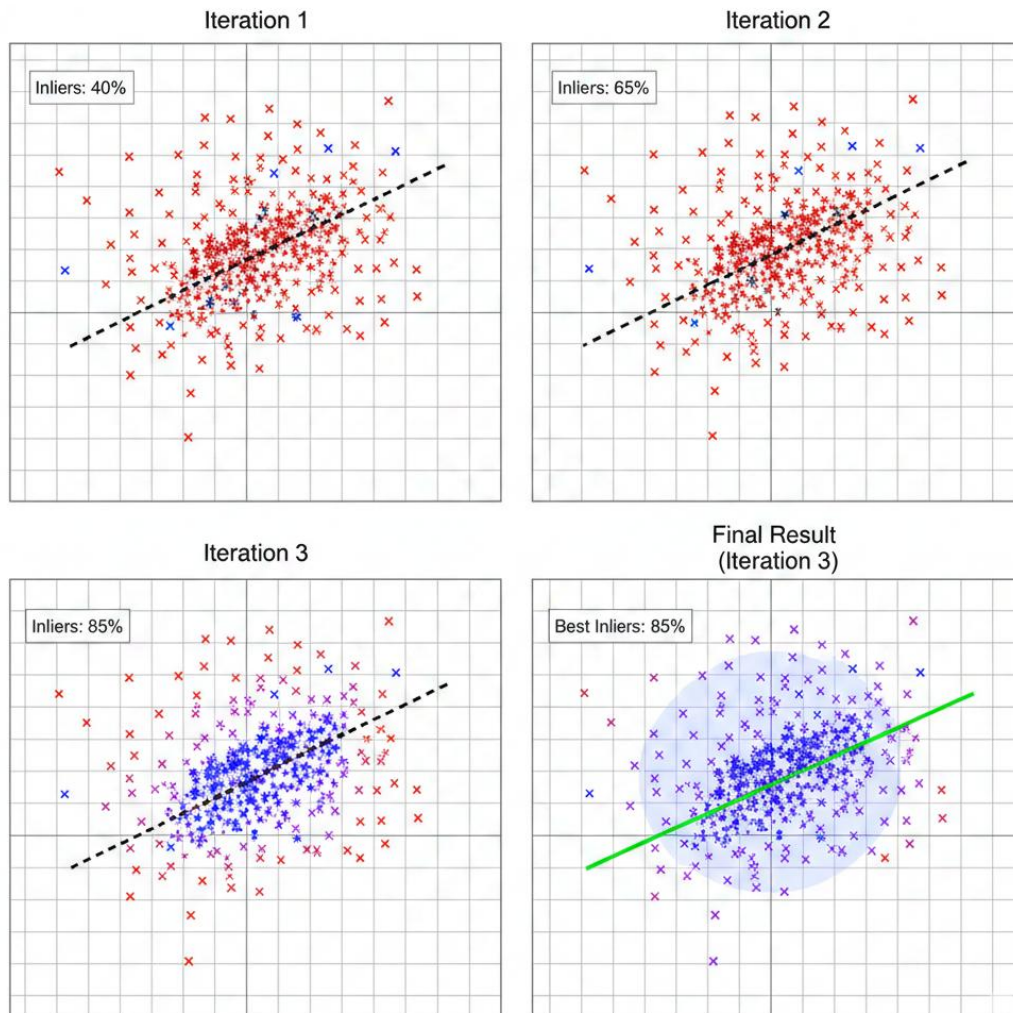
Regular Regression: Tries to fit all pixels, gets confused by noise.

RANSAC:

1. **Iteration 1:** Pick 2 random points, draw line → 40% pixels fit
2. **Iteration 2:** Pick 2 different points, draw line → 65% pixels fit
3. **Iteration 3:** Pick 2 more points, draw line → 85% pixels fit (best!)

4. **Iteration 4-100:** Continue sampling
5. **Result:** Return line from iteration 3 (most inliers)

Ignores outliers (shadow pixels, other cars) and finds the true lane.



Mathematical Equation:

RANSAC Algorithm:

1. Set: iterations k , threshold t , inlier ratio d
2. For $i = 1$ to k :
 - a) Randomly select n points (minimum needed)
 - b) Fit model M to these n points

c) Count inliers:

$$\text{inliers} = \{x \mid |y - M(x)| < t\}$$

d) If $|\text{inliers}| > d \times N$:

- Refit M using all inliers
- Save if best so far

3. Return best model

Inlier Criterion:

Point (x, y) is inlier if: $|y - f(x)| < \text{threshold}$

Number of Iterations (probability p of success):

$$k = \log(1-p) / \log(1-w^n)$$

where:

w = proportion of inliers

n = minimum points needed

p = desired success probability (e.g., 0.99)

Where:

- t = distance threshold for inliers
- d = minimum inlier ratio
- N = total data points
- k = number of iterations

14. Theil-Sen Regression

Theory:

Theil-Sen is a **robust regression** method that calculates the **median slope** from all pairs of points. It's highly resistant to outliers because it uses the median instead of mean.

- Computes slope for every pair of points
- Takes median of all slopes
- Can tolerate up to 29.3% outliers

- Computationally expensive for large datasets $O(n^2)$
- More robust than RANSAC for some cases

Real-World Example:

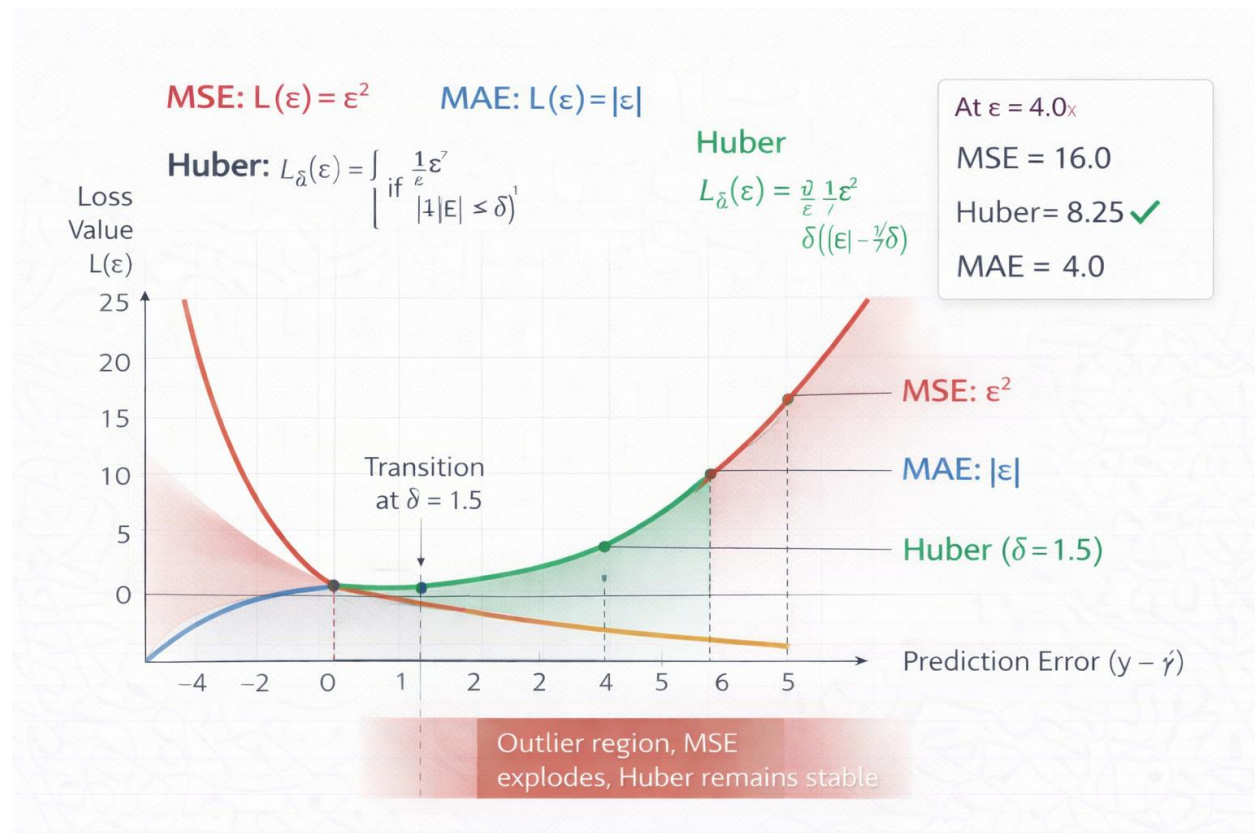
Climate Temperature Trend: Analyzing 20 years of temperature data with some measurement errors:

Regular Regression: Average slope might be skewed by a few extreme years.

Theil-Sen:

1. Calculate slope between Year 1 and Year 2
2. Calculate slope between Year 1 and Year 3
3. ...
4. Calculate slope between Year 19 and Year 20
5. Total: $(20 \times 19)/2 = 190$ different slopes
6. **Median of 190 slopes** = robust trend estimate

Even if 50 slopes are from outlier pairs, the median ignores them!



Mathematical Equation:

Theil-Sen Slope:

For all pairs (i, j) where $i < j$:

Calculate slope: $m_{ij} = (y_j - y_i) / (x_j - x_i)$

Theil-Sen Slope: $\beta = \text{median}\{m_{ij}\}$

Intercept (after finding slope):

$b = \text{median}\{y_i - \beta \cdot x_i\}$

Prediction:

$\hat{y} = \beta \cdot x + b$

Number of Slopes Computed:

Total pairs = $C(n, 2) = n(n-1)/2$

Breakdown Point:

Maximum outlier tolerance $\approx 29.3\%$

Where:

- n = number of data points
- m_{ij} = slope between points i and j
- β = median slope (robust to outliers)
- b = median intercept

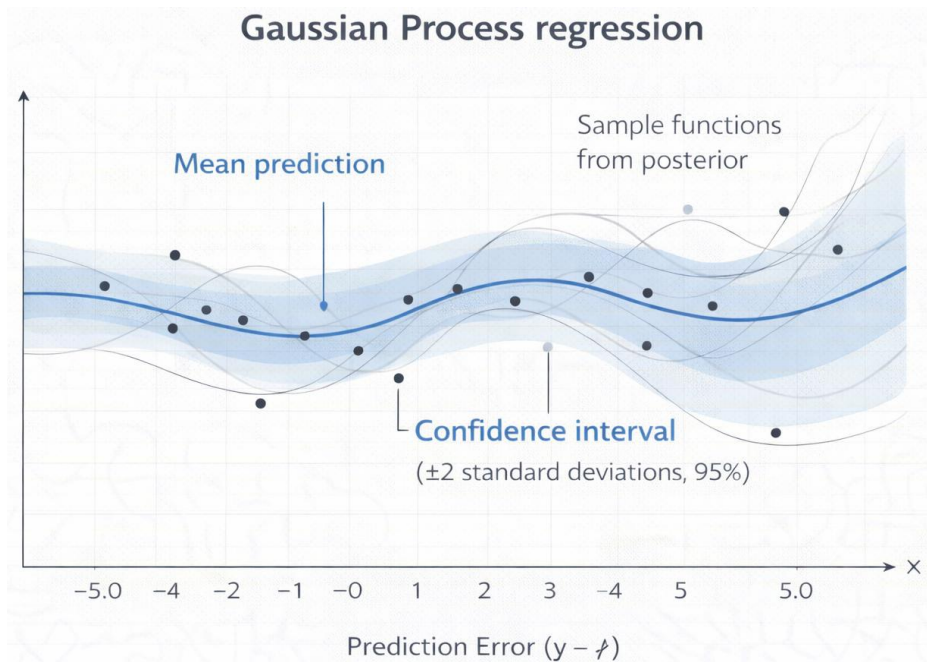
15. Gaussian Process Regression (GPR)

Theory:

Gaussian Process Regression is a **non-parametric, probabilistic** model that defines a distribution over functions. Instead of predicting a single value, it predicts a **probability distribution** with mean and variance (uncertainty).

- Every point has a probability distribution
- Captures uncertainty in predictions
- Uses kernel functions to define similarity

- Confidence increases near training data
- Computationally expensive $O(n^3)$



Real-World Example:

Robot Path Planning with Uncertainty: A robot learns terrain elevation from sensor readings:

- Near sensor readings: High confidence (narrow distribution)
- Far from sensor readings: Low confidence (wide distribution)

Example prediction:

- Point A (near data): "Elevation = 5.2m $\pm$ 0.1m" (confident)
- Point B (far from data): "Elevation = 5.0m $\pm$ 2.0m" (uncertain)

The robot can plan safer paths using this uncertainty information, avoiding high-uncertainty areas.

Mathematical Equation:

Gaussian Process Prior:

$$f(x) \sim \text{GP}(m(x), k(x, x'))$$

where:

$m(x)$ = mean function (usually 0)

$k(x, x')$ = kernel/covariance function

Posterior Predictive Distribution:

$$p(f^*|X^*, X, y) = N(f^*|\mu^*, \Sigma^*)$$

$$\mu^* = K^{*T}(K + \sigma^2 I)^{-1}y$$

$$\Sigma^* = K^{**} - K^{*T}(K + \sigma^2 I)^{-1}K^*$$

Common Kernel (RBF/Squared Exponential):

$$k(x, x') = \sigma f^2 \exp(-(\|x - x'\|^2)/(2l^2))$$

Prediction with Uncertainty:

$$\hat{y} = \mu^*(x^*) \text{ (mean prediction)}$$

$$\sigma^2(x^*) = \text{diagonal of } \Sigma^* \text{ (variance/uncertainty)}$$

Where:

- K = kernel matrix on training data
- K^* = kernel between test and training
- K^{**} = kernel on test data
- σ^2 = noise variance
- l = length scale
- σf^2 = signal variance

Summary Comparison Table

Algorithm	Type	Key Strength	When to Use	Complexity
Linear Regression	Linear	Simple, interpretable	Linear relationships	$O(n)$
Polynomial Regression	Non-linear	Captures curves	Non-linear but smooth	$O(n)$
Ridge (L2)	Regularized	Handles multicollinearity	Many correlated features	$O(n)$
Lasso (L1)	Regularized	Feature selection	High-dimensional, sparse	$O(n)$
Elastic Net	Regularized	L1 + L2 benefits	Correlated features + selection	$O(n)$
Decision Tree	Non-linear	Captures complex patterns	Non-linear, interpretable	$O(n \log n)$
Random Forest	Ensemble	Reduces overfitting	General purpose, robust	$O(n \log n \times T)$
Gradient Boosting	Ensemble	High accuracy	Competition, best accuracy	$O(n \log n \times T)$
SVR	Kernel-based	Robust within margin	Outliers, non-linear	$O(n^2-n^3)$
KNN	Instance-based	Simple, no training	Small datasets, no training	$O(n)$ per query
Bayesian Ridge	Probabilistic	Uncertainty estimates	Need confidence intervals	$O(n^2)$
Huber	Robust	Robust to outliers	Outliers in target	$O(n)$
RANSAC	Robust	Very robust (>50% outliers)	Heavy outlier contamination	$O(k \times n)$
Theil-Sen	Robust	Median-based robustness	Moderate outliers	$O(n^2)$
Gaussian Process	Probabilistic	Uncertainty quantification	Small data, need uncertainty	$O(n^3)$

The future of AI is bright, and you can be part of it!